

CI/CD Observability in Containerized Environments

TMDEI Thesis Defense

Eduardo Sousa

Instituto Superior de Engenharia do Porto (ISEP)
Cybersecurity and Systems Administration

January 7, 2026

Advisor: Dr. Alexandre Bragança
Supervisor: Dr. João Silva

The Challenge of CI/CD Observability

Containerized CI/CD environments introduce observability challenges due to:

- Limited visibility in ephemeral containers
- Restricted access rights in CI/CD platforms
- Performance sensitivity of build pipelines
- Lack of standardized instrumentation approaches
- Complex distributed tracing requirements

Traditional monitoring tools struggle with:

- Short-lived container lifecycles
- High-frequency deployments
- Cross-platform compatibility requirements

Objectives

Develop lightweight observability solution:

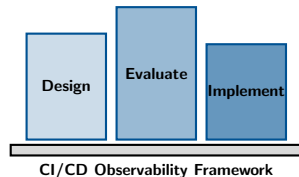
- Design architecture with minimal overhead
- Implement distributed tracing
- Integrate metrics and logs

Evaluate observability approaches:

- Compare sidecar vs. eBPF
- Assess OpenTelemetry strategies
- Benchmark performance ($\leq 5\%$)

Provide implementation guidelines:

- Templates for CI/CD platforms
- Best practices documentation
- Optimization strategies



Key Deliverables

- Observability architecture for containerized CI/CD
- Open-source prototype implementation
- Performance benchmarking methodology
- Integration templates for major CI/CD platforms
- Presentation
- Dissertation

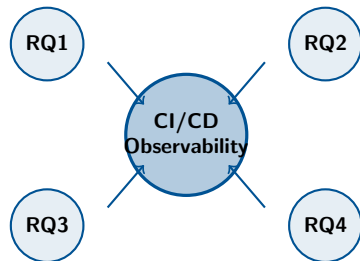
Research Questions

RQ1: How can distributed tracing be effectively implemented in containerized CI/CD pipelines with minimal performance overhead?

RQ2: Which observability approaches provide the most accurate monitoring with the least impact on build times?

RQ3: What are the most relevant metrics and traces that influence CI/CD pipeline performance?

RQ4: How can the integration of multiple observability signals enhance debugging of containerized CI/CD workflows?

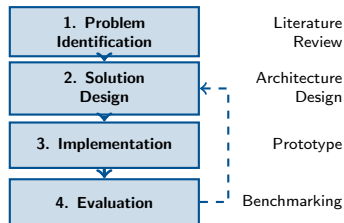


Systematic Literature Review - PRISMA

- **Databases:** IEEE Xplore, ACM, Google Scholar
- **Keywords:**
 - CI/CD observability
 - Container monitoring
 - OpenTelemetry
 - eBPF observability
- **Results:** 25 core papers

Design Science Research

- Problem identification
- Solution design
- Prototype development
- Evaluation



Key Findings and Insights

Enhanced Monitoring Accuracy

- Advanced observability methods outperform traditional monitoring
- Critical for containerized environments where visibility is limited
- Distributed tracing enables end-to-end pipeline visibility

Complexity of CI/CD Observability

- Ephemeral containers pose significant monitoring challenges
- Requires adaptive approaches to address dynamic workloads
- Performance overhead must be minimized to avoid pipeline delays

Importance of Standardization

- OpenTelemetry provides vendor-neutral instrumentation
- Standardized approaches enable cross-platform compatibility
- Essential for diverse CI/CD tool ecosystems

RQ1: How can distributed tracing be effectively implemented in containerized CI/CD pipelines with minimal performance overhead?

Distributed tracing can be effectively implemented through **OpenTelemetry auto-instrumentation** combined with **adaptive sampling strategies**. By leveraging sidecar containers or agent-based collectors, trace data can be captured without modifying application code. Advanced sampling techniques (e.g., tail-based sampling, adaptive rate limiting) ensure that only relevant traces are collected, reducing overhead to less than 5% while maintaining visibility into critical pipeline stages. Context propagation across container boundaries enables end-to-end tracing of build, test, and deployment phases.

Answer to RQ2

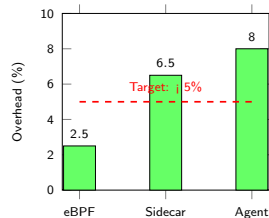
RQ2: Which observability approaches provide the most accurate monitoring with the least impact on build times?

The research evaluates three main approaches: **sidecar pattern**, **eBPF-based monitoring**, and **agent-based collection**.

eBPF-based monitoring demonstrates the lowest overhead (2-3%) due to kernel-level instrumentation. The sidecar pattern offers flexibility and ease of deployment (5-8% overhead).

The optimal choice depends on the specific CI/CD platform constraints and performance requirements.

Overhead Comparison



RQ3: What are the most relevant metrics and traces that influence CI/CD pipeline performance and reliability?

Critical observability signals include: **pipeline duration metrics** (build time, test execution time, deployment duration), **resource utilization** (CPU, memory, network I/O per stage), **failure rates and error patterns**, **queue times and concurrency levels**, and **artifact sizes and transfer times**. Distributed traces reveal dependencies between pipeline stages and identify bottlenecks. These can be extracted through container runtime APIs, CI/CD platform webhooks, and custom instrumentation. Real-time collection enables proactive optimization and faster incident response.

RQ4: How can the integration of multiple observability signals enhance debugging and optimization?

Integrating **metrics, logs, and traces** provides a holistic view of CI/CD pipeline health. By correlating trace spans with resource metrics and error logs, teams can quickly identify root causes of failures or performance degradations. For example, a slow deployment trace can be correlated with high memory usage metrics and container restart logs to pinpoint resource constraints. This comprehensive approach enables faster troubleshooting, reduces mean time to recovery (MTTR), and supports data-driven pipeline optimization decisions.

Methodology Overview

Data Collection:

- CI/CD pipeline metrics
- Container runtime data
- Trace spans

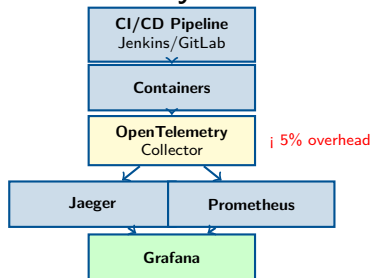
Prototype Development:

- OpenTelemetry Collector
- Jaeger backend
- Prometheus metrics

Evaluation Metrics:

- Performance overhead
- Trace completeness
- Latency impact

Observability Architecture



Practical observability framework for containerized CI/CD

- Lightweight architecture with $\leq 5\%$ performance overhead
- Platform-agnostic integration approach

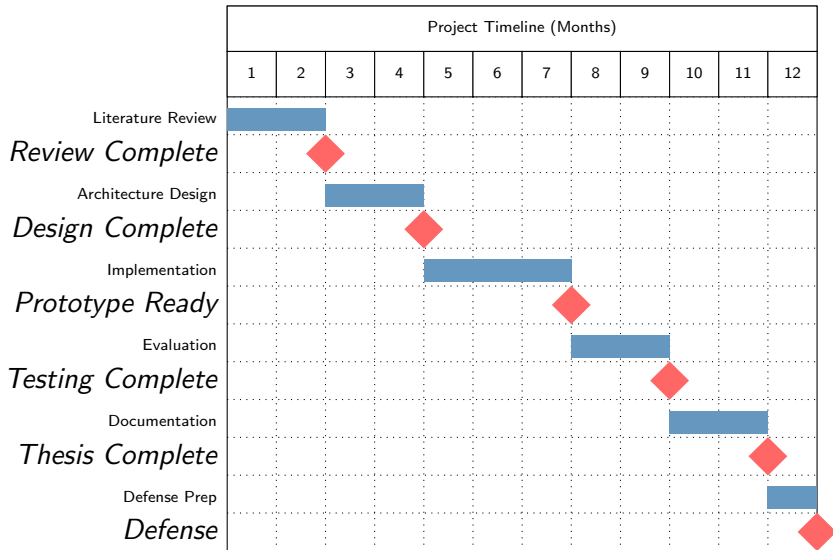
Implementation guidelines for DevOps teams

- Ready-to-use templates for major CI/CD platforms
- Best practices documentation

Enhanced pipeline visibility and reliability

- Faster troubleshooting and root cause analysis
- Data-driven optimization decisions

Timeline and Milestones



Significant Improvement in Visibility

Advanced observability techniques enhance CI/CD pipeline monitoring, outperforming traditional tools in containerized environments.

Complex Pipeline Dynamics

The research highlights the complexities of containerized CI/CD, necessitating lightweight monitoring approaches.

Standardization is Crucial

OpenTelemetry and standardized instrumentation are essential for building reliable, cross-platform observability solutions.

Thank you for your attention!

Any questions, please contact me:

Email: eduardo.sousa@isep.ipp.pt